MILES: Fast and Flexible Instruction-Level Simulation of Dense and Sparse CPU Matrix Engine

Xiaoyu Hao, Sen Zhang, Junshi Chen, and Hong An

***Abstract*—Matrix extensions (MEs) are widely adopted in modern CPUs to accelerate matrix multiplication. Architectural performance modeling of CPU MEs is challenging, which requires co-modeling the CPU, matrix accelerator (MA), and memory hierarchies. However, existing NPU simulators lack support for instruction-level simulation, accurate CPU modeling, or flexible accelerator model customization. To this end, this paper presents MILES, an instruction-level simulation framework for dense and sparse matrix engines in CPUs. MILES is a trace-driven and cycle-level simulator. MILES introduces a memory-centric performance model, MCPM, which unifies performance models of several dense and sparse MAs. By simulating memory operations in detail and simplifying arithmetic operations, MCPM achieves high simulation accuracy while maintaining high speed. MILES also proposes a memory-centric accelerator representation, MCAR, to support flexible customization of accelerators through external descriptions and implements several sparse MAs based on it. Experimental results show that MILES achieves average errors of 9% and 10.5% for OS and WS systolic arrays, respectively, compared with BOOM+Gemmini RTL simulation, while providing a $432\times$ simulation speedup. For sparse accelerators based on Gustavson, inner-product, and outer-product dataflows, MILES achieves errors of 11.2%, 8.6%, and 3.8%, respectively, compared with sstStonne, while obtaining a $20\times$ simulation speedup. These results demonstrate that MILES provides an accurate, efficient, and flexible framework for CPU–accelerator co-simulation of CPU MEs.**

***Index Terms*—CPU, Matrix Extension, Accelerator, Simulator, Matrix Multiplication.**

I. Introduction

In recent years, with the rapid development of artificial intelligence and scientific computing, matrix extensions (MEs) have been widely adopted in mainstream instruction set architectures (ISAs) [1]–[6]. Compared with scalar and vector instructions, matrix instructions (MIs) are more complex, involving both arithmetic and memory operations. Their performance depends not only on the matrix accelerator (MA) but also on the CPU and the memory system. Therefore, in the early design stage, it is crucial to co-model these components to explore the design space. However, building a simulator for MEs faces three challenges.

First, modeling sparsity. The primary goal of ME is to accelerate General Matrix Multiplication (GEMM). Meanwhile, leveraging sparsity to improve the performance of MAs has been widely studied [7]–[10]. However, sparse MAs exhibit indirect memory access that depends on the input data, necessitating a different modeling approach than dense MAs. Second, accurate co-modeling of CPU, MA, and the memory system is complex. MIs and general-purpose instructions share CPU pipeline resources, so their scheduling, dependencies, and resource contention can affect the performance of running matrix operations. In addition, shared memory resources such as caches and DRAM must be modeled to capture memory interference between the CPU and MA. Third, the conventional instruction-level cycle-by-cycle simulation approach, while achieving high accuracy, suffers from slow simulation speed. Additionally, MIs are difficult to model as a fixed latency, as is commonly used for scalar instructions, due to their complex memory and arithmetic operations, which causes simulation cost.

Recently, many cycle-level Neural Processing Unit (NPU) simulators have been proposed, but they still have the following three shortcomings for CPU ME simulation: (1) Lack of accurate CPU performance models, especially for Out-of-Order (OOO) execution. Currently, very few works, like PytorchSim [11] and SMAUG [12], can simulate CPUs. (2) Lack of instruction-level simulation (ILS) methods. Due to the absence of compiler support, some works [13]–[15] use custom input formats and implement cycle-level simulators driven by compute and memory events. Without instruction-level representation, they achieve significant errors when simulating MIs. Although PytorchSim supports MIs in Gem5 [16] and Spike [17], it only collects latency from them as input to its performance model, rather than co-simulating the CPU and MA. Also, SMAUG invokes accelerators via system calls rather than instructions. (3) Lack of flexible representation for accelerators. Most existing approaches hard-code accelerator models inside the simulator. Introducing new architectures or modifying existing models often requires additional development effort. Only a few simulators [12], [18], [19] support customizing accelerator models through external descriptions, i.e., C kernels.

To address these limitations, we propose MILES[1], an ILS framework for dense and sparse MEs in CPUs. MILES is built on CISim [20], which provides accurate trace-driven, cycle-level modeling of OOO CPUs and memory hierarchies. MILES extends CISim with a Gemmini-like coprocessor model [21] to support MIs. It uses LLVM IR as the ISA, where general instructions are executed on the CPU model, and MIs are dispatched to the coprocessor model. MAs are simulated based on a memory-centric performance model (MCPM). MCPM abstracts execution of several GEMM and sparse-sparse matrix multiplication (SpMSpM) accelerators

Xiaoyu Hao, Sen Zhang, Junshi Chen, and Hong An are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China. (E-mail: haoxiaoyu@mail.ustc.edu.cn)

[1]MILES is named after: **M**atrix, **I**nstruction-**L**evel, **E**ngine, and **S**imulator

at the vector and element levels. Based on MCPM, MILES also introduces a memory-centric accelerator representation (MCAR), which allows new accelerator models to be flexibly defined outside the simulator implementation as C-language kernels with address stream markers and synchronization primitives. MCPM is implemented based on a Memory Request Generator (MRG), which reads address streams and replays memory requests to achieve cycle-level simulation of MAs. Based on MCPM and MCAR, MILES currently supports modeling of systolic arrays (SAs), with Output Stationary (OS) and Weight Stationary (WS) dataflows [21], as well as representative dataflows of SpMSpM accelerators including: (i) Gustavson-based and (ii) Outer Product-based Flexagon [10], abbreviated as Gust and OP, respectively, and (iii) Inner Product-based Sigma [8]. In summary, this paper makes the following contributions:

- We develop MILES, an ILS framework that enables fast and flexible co-simulation of CPU, MAs, and memory hierarchies at the system level.
- We propose MCPM, a unified memory-centric performance model for several GEMM and SpMSpM accelerators of representative dataflows. It enables fast and accurate modeling of MAs by simulating memory operations in detail and simplifying computational operations.
- We propose MCAR to flexibly define accelerator models in external descriptions (C-language kernels) without modifying the simulator.
- Validation against RTL simulation of BOOM [22]+Gemmini [21], the errors of MILES for OS and WS SAs are only 9% and 10.5%, respectively, while achieving a simulation speed 432 times faster than the 8-thread RTL simulation.
- Validation against sstStonne [23], MILES achieves a maximum performance error of only 11.2% across multiple SpMSpM accelerators, while providing a 20× speedup.
- We demonstrate the usefulness of MILES through a case study on memory optimization for a SpMSpM accelerator, in which a Hybrid Buffering Scheme (HBS) is proposed and achieves a 1.22× average speedup.

II. Background and Motivation

A. The Importance of CPU–Accelerator Co-Modeling

MEs in mainstream ISAs usually rely on dedicated architectural matrix registers, such as AMX Tile registers [1] and SME ZA registers [3], to provide the data-level parallelism required by GEMM. In such designs, MIs can directly encode source and destination operands via architectural register state. Another class of designs, represented by Gemmini [21], integrates the ME as a coprocessor and uses scratchpad memory (SPM) as the main on-chip storage. In these SPM-based designs, MIs often need to explicitly encode DRAM/SPM addresses, data sizes, offsets, and other configuration parameters. As a result, a single high-level matrix operation is usually decomposed into multiple MIs and run in loops. Scalar instructions on the CPU generally generate the corresponding addresses and parameters, which may delay the dispatch of MIs to the coprocessor, leading to the configuration wall [24]. This makes instruction-level CPU–accelerator co-modeling particularly important for SPM-based MEs.

Fig. 1 illustrates this effect by breaking down cycles when Gemmini is integrated with Rocket [25] and BOOM [22], respectively. Gemmini uses the default 16×16 SA configuration. Experiments are conducted using RTL simulation, and cycles spent on CPUs and Gemmini are measured using performance counters. Results show that BOOM not only reduces CPU cycles, but also reduces Gemmini-side elapsed cycles. This is because a more powerful CPU can generate configuration parameters and dispatch MIs more efficiently, thereby reducing the time the coprocessor waits for MIs. For example, in the 512×512 case, the CPU and Gemmini execute about 1.47 million instructions in total, whereas only about 32 thousand are GEMM instructions. Even when data movement instructions such as `mvin` and `mvout` are included, MIs still account for only a small fraction of the total instruction count. The large number of scalar instructions can therefore delay the dispatch of MIs, especially when address generation involves long-latency operations such as `div` and `rem`. This result highlights the need for instruction-level CPU–accelerator co-modeling for designing SPM-based MEs.

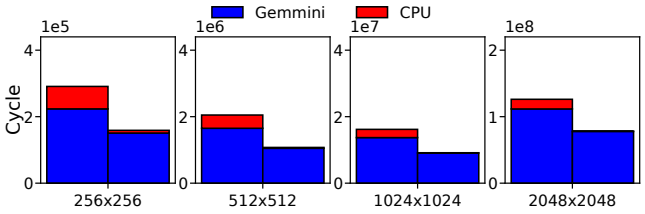


Fig. 1: Cycle breakdown of Gemmini with Rocket and BOOM

B. Related Works and Limitations

Although CPU simulators such as Gem5 [16] are capable of providing high simulation accuracy and already support certain MEs, such as ARM SME [3], modeling new MAs of various execution models and dataflows, such as Gemmini implemented as a coprocessor [21], often requires building microarchitecture models from scratch, which is not flexible. Currently, NPU simulators have been extensively studied. Analytical models [26]–[28] enable fast exploration of design space and computational mapping, but cannot simulate microarchitectural details. In contrast, cycle-level NPU simulators provide higher accuracy, but as shown in Table I, they still have several limitations for CPU MEs.

First, many NPU simulators, such as ONNXim [15], mNPUSim [14], and SCALE-Sim [29], lack CPU models. Accurately modeling OOO CPU cores is complex and requires an instruction-level representation and a dedicated performance model. Currently, PytorchSim [11] and SMAUG [12] can simulate CPUs based on Gem5 [16]. Second, as discussed previously, ILS of the CPU and accelerator is essential for ME design. However, PytorchSim uses a tile-based performance model instead of ILS. It enables RISC-V MIs but relies on too many toolchains, including Gem5 [16], Spike [17], and

TABLE I: Limitations of current NPU simulators

	Instruction-Level Co-Simultion	Accurate CPU Model	Sparsity	Customizable Behavior
PytorchSim [11]	✗	△	✓	✗
sstStonne [23]	✗	✗	✓	✗
SCALESimV3 [13]	✗	✗	✓	✗
mNPUSim [14]	✗	✗	✗	✗
ONNXim [15]	✗	✗	✗	✗
SMAUG [12]	✗	✓	✗	✓
MILES	✓	✓	✓	✓

△: Partial Support

MLIR [30], making it costly to add new instructions. SMAUG can co-simulate CPUs and accelerators, but it does not control accelerators with instructions. Meanwhile, although SMAUG supports SAs, its array size cannot exceed 8×8, and it does not support sparsity modeling. Third, among the above works, only SMAUG supports customizing accelerator models with external descriptions. Simulators such as Gem5-Aladdin [18] and Gem5-SALAM [19] require the whole program written in C kernels to model standalone accelerators. TeAAL [31] provides a domain-specific language for generating simulators of SpMSpM accelerators, without support for CPU modeling. In contrast, the proposed MCAR requires only the description of key memory operations designed specifically for MAs.

C. Toward a Unified Abstraction for Matrix Accelerator

MILES builds performance models for representative dataflows used in GEMM and SpMSpM accelerators. For GEMM, MILES models two typical dataflows of SAs: WS and OS, which have been adopted in Google TPU [32] and Gemmini [21]. For SpMSpM, MILES models three academic accelerators with representative dataflows, including the Flexagon of Gustavson (Gust) and the outer product (OP), and Sigma [8] with an inner-product dataflow. Table II summarizes the dataflow types they use and their computational behaviors at each phase. GEMM Execution models of these GEMM and SpMSpM accelerators can be unified at the vector level.

As shown in Fig. 2, for SpMSpM accelerators, input matrices are usually compressed into formats such as CSR or CSC. During execution, the accelerator first reads contiguous nonzero elements from A as a stationary vector in the stationary (STA) phase. Then, data vectors gathered from B are fed into the accelerator during the streaming (STR) phase. Since these elements are usually accessed indirectly, they may be non-contiguous in memory, leading to irregular memory accesses. Spatial tree-based reduction [8] logic is generally used to perform multiply-and-add operations on these vectors to generate partial sums.

Execution of GEMM accelerators can also be viewed as vector-level operations on two input matrices. For example, with a WS SA, weight data is gradually loaded into the array at the vector granularity and reused. Then, the activation matrix is continuously streamed into the array at the vector granularity to trigger computation. For an OS SA, vectors from the two input matrices stream into the array simultaneously, while partial sums are kept and accumulated inside the array. All memory access operations in SA are data movements between SPM and computational logic, excluding data transfers between the SPM and other memory levels.

The writeback (WB) phase of the aforementioned SAs can likewise be abstracted at the vector or element level. Specifically, partial sums generated by the WS SA propagate downwards through the array and are written back row by row in vector form. In contrast, the OS dataflow stores and accumulates partial sums in the array until the STR phase is complete, at which point it outputs them row by row. The Gust accelerator's WB also overlaps with the STR, producing a single row of output at a time. The Sigma accelerator exhibits similar write-back behavior, but, as its STA phase can hold multiple rows of data, it may output multiple rows of results. The OP accelerator generates partial sums during the STR and temporarily stores them in Intermediate Memory (IM). These partial sums are written back to memory only after they have been merged by row. In these SpMSpM accelerators, data is input from the leaf nodes of the multiply-accumulate tree and output from the root node after reduction. As the root node can produce only one result per cycle, partial sums are written back at the element level.

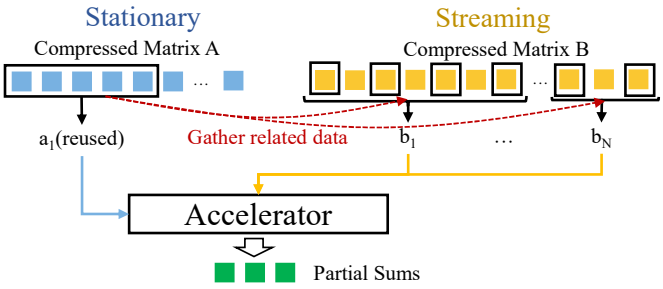


Fig. 2: Vector-level abstraction of representative SpMSpM accelerators

III. MILES Overview

MILES has developed a trace-driven, cycle-level simulator that uses LLVM IR as its ISA. Building on CISim, MILES implements a coprocessor microarchitecture model, similar to Gemmini [21], to support the simulation of MIs. As shown in Figure 3, MILES's inputs include the workload's LLVM IR and hardware configuration. MILES instruments and collects traces of general-purpose instructions, which are subsequently executed by the CPU model. When the CPU model encounters an MI, that instruction is dispatched to the coprocessor model for execution. The coprocessor model employs a Memory-Centric Performance Model (MCPM) to model various MAs.

The MCPM is implemented via a Memory Request Generator (MRG). For GEMM accelerators, including OS and WS SAs, the MRG can directly generate vector-level memory access requests based on parameters such as the address, data size, and matrix dimensions encoded in the MIs. MILES further provides a Memory-Centric Accelerator Representation (MCAR) to represent accelerator behavior. In this way, MILES has modeled SpMSpM accelerators such as Gust, OP, and Sigma. The MCAR-based SpMSpM modeling process comprises three stages: first, the user describes the accelerator's

TABLE II: Matrix accelerators modeled by MILES

Accelerator	Stationary	Streaming
Gust	Nonzero elements of one sparse row of A: $[A_{i,k_1}, A_{i,k_2}, \ldots, A_{i,k_m}]$, where k_r is the column index of a nonzero element in the i-th row of A	Nonzero elements in rows of B selected by the column indices k_r from the stationary vector form streaming input vectors: $\mathbf{b}^{(t)} = [B_{k_r,j_{r,t}}]_{r\in\mathcal{R}_t}$. Here, $j_{r,t}$ is the column index of the t-th nonzero element in the k_r-th row of B, $\mathcal{R}_t = \{r \mid t \le n_r\}$ is the set of row indices that still contain valid nonzero elements in the t-th streaming vector, and n_r is the number of nonzero elements in the k_r-th row of B
Sigma	Nonzero elements of one or multiple sparse rows of A	Columns of B are streamed one by one, where the nonzero elements are indexed by the nonzero columns of all stationary rows in A
OP	Nonzero elements of one or multiple sparse columns of A with column indices $k_1, \ldots, k_q$	The input vector is: $\mathbf{b}^{(t)} = [b_1^{(t)}\mathbf{1}_{m_1}; \ldots; b_q^{(t)}\mathbf{1}_{m_q}]$, where m_r is the number of nonzero elements in the k_r-th column, $b_r^{(t)} = B_{k_r,j_{r,t}}$ is the t-th nonzero element taken from the k_r-th row of B, $j_{r,t}$ is its column index, and $\mathbf{1}_m$ denotes an all-ones vector of length m

In the Gust and OP accelerators, the superscript (t) denotes the t-th streaming input vector.

core memory operations in a C-language kernel function using address stream labels and synchronization primitives. Secondly, MILES uses the LLVM toolchain to instrument this function. During execution, it collects the marked memory addresses, writes them to their respective address-stream files, and simultaneously records the execution order of the address streams. Subsequently, during the simulation, MRG replays these memory requests using the execution order file and address stream files.

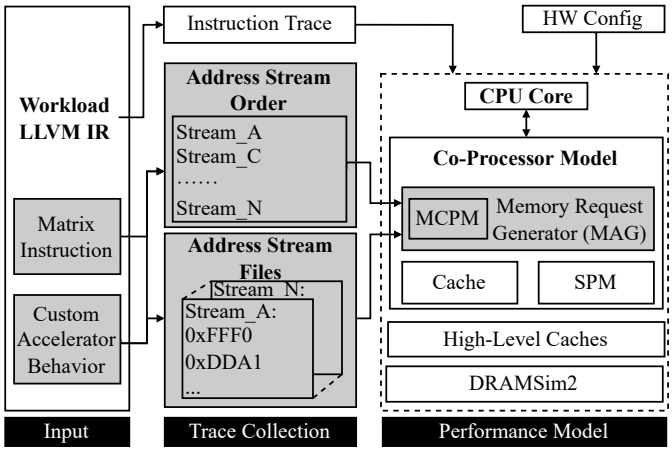


Fig. 3: The workflow of MILES

IV. Memory-Centric Performance Model

As discussed in Section II-C, although MAs differ in terms of data flow, storage formats, and on-chip memories, their execution processes can still be uniformly abstracted at the vector or element level. To construct an accurate and fast cycle-level performance model, it is necessary to determine when each memory access request is initiated, when the requested data becomes available, how the data is utilized in computations, and when results are written back. Consequently, this paper proposes a Memory-Centric Performance Model (MCPM). By modeling memory operations in detail and simplifying computational operations, the MCPM accelerates simulation speed while maintaining accuracy, enabling unified modeling of various GEMM and SpMSpM accelerators.

A. Unified Performance Model

Fig. 4 illustrates MCPM's unified modeling of the execution processes of various matrix accelerators. For a single GEMM or SpMSpM operation comprising n data vector requests, let v_i denote the ith data vector being processed. MCPM defines four types of delay parameters for v_i: I_i denotes the issue interval between v_i and v_{i-1}, R_i denotes the memory access delay for v_i, C_i denotes its processing time, and W_i denotes the write-back delay for its results. In particular, for SpMSpM accelerators that utilize a cache, R_i and W_i are cache access latency, whereas for SAs that utilize SPM, they represent only the SPM access latency. The latency of data transfer between SPM and other memory hierarchies is modeled using explicit data-moving instructions and DMA operations.

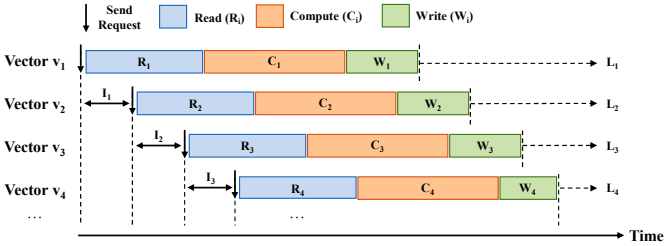


Fig. 4: MCPM for matrix accelerator

Equation 1 shows the time that the ith vector memory access request must wait before being issued. Here, $I_0 = 0$ indicates that the first request is issued immediately upon the start of MI execution. For $k \ge 1$, I_{k-1} denotes the time interval between the kth and the $(k-1)$th data vector requests.

$$T_i = \sum_{k=0}^{i-1} I_k, \quad i \ge 1 \tag{1}$$

Equation 2 shows L_i, the time at which the computation of the ith data vector is completed. Here, R_i, C_i, and W_i correspond to the read, computation, and write-back latencies for that data vector, respectively. During the STA phase, data vectors are typically only read into the computational logic and temporarily stored. Consequently, no write-back operations occur. In this case, $W_i = 0$, and C_i denotes the propagation delay of data within the on-chip network. Since matrix multiplication requires processing all data vectors, and

the MA executes the same phase sequentially for different vectors, the completion time of the matrix multiplication can be approximated by the completion time L_n of the last vector.

$$L_i = T_i + R_i + C_i + W_i = \sum_{k=0}^{i-1} I_k + R_i + C_i + W_i, \quad i \geq 1 \quad (2)$$

Based on this vector-level latency model described above, we now outline methods for calculating write-back request timing, memory access and computational latencies, and request intervals.

B. Modeling When Store Request is Issued

In Equation 2, store requests are issued after the ith data vector has been fully processed, and the delay is denoted by W_i. However, the timing of issuing store requests varies across accelerators and does not always commence immediately upon completion of the computation, as summarized in Table III. Write-back can be broadly categorized into two types. The first type has an independent WB phase, in which store requests typically must wait to issue until all output data has been fully generated. For example, OS SA needs to wait for partial sums to propagate to the bottom of the array after the STR phase has ended before it can generate the corresponding store requests. As the OS SA typically supports overlapping STR and WB phases via hardware double-buffering, it stores the first vector precisely when the last input vector is processed. The OP accelerator, on the other hand, must first merge the partial sums after the STR before storing results. Since partial sums are obtained via a reduction network, MCPM can simplify the delay of merging by treating it as part of C_i, which is modeled as the depth of the reduction network.

TABLE III: Memory access time of representative GEMM and SpMSpM accelerators

Name	Stage	Actual Memory Access Time
OS SA	WB	$2d-1$ cycles after all inputs are readys
WS SA	STR	$2d-1$ cycles after each input vector is ready
Gust	STR	After each vector completes computation
OP	WB	When a partial sum has been merged into the final output
Sigma	STR	After each vector completes computation

The second type does not have a separate write-back phase. Instead, WS SA, Gust, and Sigma accelerators store results during the STR phase. For a $d \times d$ WS SA, during the STR phase, a store request for the corresponding result is generated $2d-1$ cycles after the memory access of an input vector completes, which coincides precisely with the time it takes for that vector to flow from left to right through the array. For the Gust and Sigma accelerators, MCPM employs an existing design [10] that first writes partial sums to an IM and then stores the results once the STR phase ends. By introducing a WB phase, MCPM avoids the need to track and model the timing to write back each result, thereby reducing modeling complexity.

C. Element-Level Performance Modeling

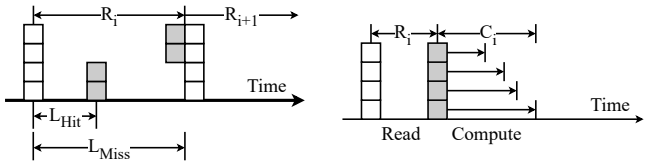
SpMSpM accelerators typically take compressed matrices as input. Due to indirect memory access via indexing, the addresses of elements within a data vector may be non-contiguous in memory. Consequently, the accelerator issues these memory requests element-by-element. As shown in Figure 5a, since the access latency for these elements may vary, MCPM uses Equation 3 to calculate the vector-level access latency R_i. Here, D_i is the set of data elements in the i-th data vector, x denotes one element within it, and $r(x)$ denotes its reading latency. For GEMM accelerators with regular memory access, such as OS and WS SAs, MCPM directly issues vector-level memory requests, with a reading latency of R_i.

$$R_i = \max_{x \in D_i} r(x) \quad (3)$$

Similarly, the computation time for different elements within the same data vector may also vary. Taking SA as an example, input data is fed into the array in a skewed manner via synchronous FIFOs. Consequently, as shown in Figure 5b, for data input from the left, the computation time increases row by row from top to bottom, whilst for data input from the top, it increases column by column from left to right. In this case, the vector-level computational delay can be expressed by Equation 4.

$$C_i = \max_{x \in D_i} c(x) \quad (4)$$

Here, $c(x)$ denotes the propagation and computation time of x within the computational logic. Since MCPM employs a synchronous execution model, computation cannot begin until all elements in the vector are ready. In this case, $c(x)$ can be further simplified to a fixed delay c, and $C_i = c$. Specifically, for OS and WS SAs of size $d \times d$, c is $2d-1$, which represents the maximum delay for data computation within the SA. The computational logic of the SpMSpM accelerators modeled in this paper consists primarily of a distribution network and a reduction network. For a distribution network employing an N-input, N-output non-blocking topology [8], its delay can be expressed as $c^{\text{dist}} = 2 \times \log(N) + 1$. For a reduction network with M multiplication units, its delay is $c^{\text{red}} = \log(M) + 1$. In summary, the computational delay of an element can be expressed as: $c = c^{\text{dist}} + c^{\text{red}}$



(a) Memory access (b) Computation

Fig. 5: Fine-grained memory access and computation latency

Write-back latency is also modeled at the element level. Let O_i denote the set of data elements that need to be written back to the memory system, and let $w(x)$ denote the time required

for element x to be successfully issued to the memory system. Then the write-back latency for the ith data vector is:

$$W_i = \max_{x \in O_i} w(x) \tag{5}$$

A store operation is marked complete when it is successfully issued to the memory system. However, if structural hazards such as cache port contention or SPM bank conflicts occur, the issue will be blocked, thereby increasing $w(x)$.

D. Modeling Memory Request Intervals

The issue interval I_i between data vectors is also critical to model accelerator performance. MCPM also uses a synchronous model to issue read requests. The request for the $i+1$th data vector must wait until the requests for the ith data vector have been completed, so the issue interval can be approximated as:

$$I_i = R_i, \quad i \geq 1 \tag{6}$$

When accelerators use pipelined SPM as their on-chip memory, memory requests typically do not need to wait for the previous requests to complete before being issued. Ideally, the interval between consecutive requests is one cycle, i.e. $I_i = 1$. However, in practice, the issue interval between adjacent requests should not be 1. Additional blocking cycles B_i caused by bank conflicts, port contention, or other architectural hazards should be taken into account:

$$I_i = 1 + B_i, \quad i \geq 1 \tag{7}$$

$r(x)$, $w(x)$, and B_i are defined with respect to the read, write-back, and issue delays, respectively. MCPM uses these variables to describe how to evaluate the execution latency of an MA at the vector and element levels. As MILES aims to develop an accurate cycle-level performance modeling method, all three delays are measured through cycle-level state updates and event simulation in implementation.

E. Modeling Pipelined Accelerator

MCPM models pipelined accelerators as follows: once reading requests for data vector $i-1$ are complete, the subsequent data vector i can begin execution. As computational granularity can vary across MIs, an MI may contain one or more STA, STR, and WB phases. Consequently, within an instruction, the next hardware phase will not commence until the read request for the final vector of the preceding phase has been completed. Between instructions, the next MI will not begin execution until the read request for the final vector of the preceding instruction has been completed.

V. Memory-Centric Accelerator Representation

MILES proposes a Memory-Centric Accelerator Representation (MCAR). MCAR defines the execution of an accelerator as a C-language kernel and characterizes its memory operations by address stream markers and synchronization primitives. Based on MCAR, MILES implements three SpMSpM accelerator models: Gust, OP, and Sigma. During simulation, MILES treats MCAR kernels as an MI. Address stream markers and synchronization primitives within the function body are used solely to describe an accelerator's memory operations and are never executed in the simulator.

A. Address Stream Marking

MCAR provides a macro `MARK_STREAM` to mark memory instructions used to describe an accelerator's memory operations, which encapsulates the annotation function `__builtin_annotation`. MILES only instruments `load` or `store` marked with `MARK_STREAM` and ignores all other instructions. During trace collection, whenever MILES encounters a marked memory instruction, it writes the memory address to a file named after the corresponding address stream tag and records that tag in an address stream order file. MILES replays addresses in simulation following this order file. Furthermore, MCAR also supports the `STREAM_NAME:bytes:delay` syntax. Here, `bytes` denotes the bytes required by a memory request, and `delay` indicates the number of cycles that a request should wait before being issued to the memory system. `delay` is used to model the latency a store request should wait before issuing, until its relevant computation is finished. During address stream collection, MILES records `bytes` and `delay` after the corresponding address in address streams, thereby enabling the simulator to read and control how an accelerator generates memory requests.

B. Synchronization Primitives

MCAR also provides a set of synchronization primitives to describe the synchronization relationships between different address streams. The function `MILES_wait_load` suspends the generation of subsequent memory requests until the data for all preceding read requests is ready. The function `MILES_wait_store` suspends the generation of memory requests until all preceding write requests have been issued. The function `MILES_ci_finish` indicates the end of the memory request generation for an MI and must be placed at the end of an MCAR kernel. When MILES encounters primitives during trace collection, it writes corresponding numerical markers to the address stream order file. In practice, values -2, -3, and -1 correspond to the above primitives, respectively. Simulator pauses request generation, synchronizes, and finishes accelerator execution based on these markers.

Synchronization primitives and address stream markers enable the loose description of vector-level memory operations. For SpMSpM accelerators, a vector typically consists of multiple elements with non-contiguous addresses. To this end, MCAR can represent a vector memory operation via multiple discrete memory requests marked with `MARK_STREAM` and delimit vector boundaries using `MILES_wait_load` or `MILES_wait_store`. In this way, MCAR supports both the vector-level abstraction required by MCPM and the element-level irregular memory operations resulting from sparse indices and compressed formats in SpMSpM.

C. MCAR Kernels of SpMSpM Accelerators

This section presents MCAR kernels of three representative SpMSpM accelerators. In these kernels, `STREAM_A`, `STREAM_B`, and `STREAM_C` are stream tags for stationary matrix A, streaming matrix B, and output matrix C, respectively. Furthermore, `MS` is used to denote `MARK_STREAM`.

Algorithm 1: MCAR of the Gust Accelerator

```
1  Function GUSTAVSON-MCAR(A (CSR), B (CSR), C):
2      nnz ← 0
3      for i ← 0 to M − 1 do
4          IM ← 0                          // An internal buffer
5          foreach nonzero-element block S_A do
6              read and store (k, val_A) to S_A, and mark val_A
                 with STREAM_A                          // STA
7              MILES_wait_load()
8              foreach (k, val_A) ∈ S_A do
9                  cursor[k] ← B.rowptr[k];
                     end[k] ← B.rowptr[k + 1]          // STR
10             while there exist unfinished B rows do
11                 foreach (k, val_A) ∈ S_A do
12                     if cursor[k] ≥ end[k] then continue
13                     bp ← cursor[k]++
14                     j ← B.colidx[bp]
15                     val_B ←
                         MS(&B.values[bp], "STREAM_B")
16                     IM[j] ← IM[j] + val_A × val_B
17                 MILES_wait_load()
18         for j ← 0 to N − 1 do
19             if IM[j] ≠ 0 then
20                 MS(&C.values[nnz +
                     +], "STREAM_C") ← IM[j]          // WB
21         MILES_wait_store()
22     MILES_ci_finish()
```

Algorithm 2: MCAR of the Sigma Accelerator

```
1  Function SIGMA-MCAR(A (Bitmap), B^T (Bitmap), C):
2      i ← 0
3      while i < M do
4          (R, K, V_A) ← read nonzero elements from A no
             more than X, and mark the corresponding A value
             reads with STREAM_A                        // STA
5          MILES_wait_load()
6          startK ← min(K); endK ← max(K)               // STR
7          for j ← 0 to N − 1 do
8              B_j ← 0
9              for kk ← startK to endK do
10                 if B^T.bitmap[j][kk] = 0 then continue
11                 idx ← ValueIndex_{B^T}(j, kk)
12                 val_B ←
                     MS(&B^T.values[idx], "STREAM_B")
13                 if kk ∈ K then B_j[kk] ← val_B
14             foreach r ∈ R do
15                 IM[r, j] ←
                     IM[r, j] + Σ_{kk∈K} V_A[r, kk] × B_j[kk]
16             MILES_wait_load()
17         write back matrix C and mark writeback requests
             with "STREAM_C"                            // WB
18         MILES_wait_store()
19         i ← next unprocessed row
20     MILES_ci_finish()
```

Gust accelerator: MCAR kernel of Gust accelerator is shown in Algorithm 1. Matrix A is processed row by row, and partial sums are buffered in an intermediate memory IM. Nonzeros of the i-th row of A are tiled into blocks S_A whose size is the number of multipliers X. In the STA phase, Gust reads the nonzero elements (k, val_A) from A and stores them to S_A and marks load instructions of val_A with `STREAM_A`. Then, in the STR phase, Gust reads the k-th row of matrix B according to the column index k of each nonzero element in S_A, and initializes a cursor for that row as $cursor[k] = B.rowptr[k]$ and $end[k] = B.rowptr[k+1]$. In each iteration, Gust checks the B rows involved in S_A one by one. If the current cursor has not reached the end, Gust reads a nonzero element, marks it as a `STREAM_B` address stream, and then increments $cursor[k]$. Then that element is multiplied by the corresponding val_A and accumulated into IM. This process continues until all B rows end. Finally, in the WB phase, results in IM are written back to matrix C with store instructions marked with `STREAM_C`. Nonzero elements of C are compressed into contiguous addresses, with the variable nnz indicating the offset.

Sigma accelerator: MCAR kernel of Sigma accelerator is shown in Algorithm 2. Sigma first constructs a stationary vector from A with no more than X nonzero elements, and marks the corresponding load instructions with `STREAM_A`. Here, $\mathcal{R}$ and $\mathcal{K}$ contain row and column indices of these elements, and V_A records their values. When the number of non-zero elements in the ith row of matrix A exceeds X, that row is divided into several blocks of size X, with each stationary vector corresponding to one of these blocks. When the number of non-zero elements in a single row is less than X, the non-zero elements from several adjacent rows are concatenated to form a stationary vector of length no greater than X. The STR phase first computes $startK = \min(\mathcal{K})$ and $endK = \max(\mathcal{K})$, which define the index range for gathering nonzeros of the rows of B^T. For the j-th row of B^T, when $B^T.bitmap[j][kk]$ is true, Sigma reads the corresponding non-zero element and marks its load instruction via `STREAM_B`. If kk belongs to $\mathcal{K}$, the element is buffered in B_j to perform MAC operations with the stationary vector later. The WB phase stores results in IM back to DRAM in dense format, and marks store instructions via `STREAM_C`, which can generate irregular address accesses.

OP accelerator: MCAR kernel of OP accelerator is shown in Algorithm 3. OP first reads a stationary vector S_A with at most X nonzero elements from matrix A, where each nonzero element is represented as (i, r, val_A). Here, i and r denote the row and column indices of the element in matrix A, respectively, and the load instruction of val_A is marked with `STREAM_A`. Then, the STR phase initializes cursors for the rows of matrix B by $cursor[r] = B.rowptr[r]$ and $end[r] = B.rowptr[r+1]$. For each nonzero element in S_A whose column index r_A equals the current B row index r_B, the algorithm reads $B[r_B, j]$ once and marks its instructions with `STREAM_B`. These loaded val_B values jointly form a streaming vector, as shown in Lines 12–19. The multiplication operations generate multiple partial sums located in column j of matrix C but in different rows, and these results are stored in $\mathcal{P}$. This process continues until all B rows finish, that is, cursors reach the end. Finally, in the WB phase, partial sums in $\mathcal{P}$ are first merged, and results are then written back to

Algorithm 3: MCAR of the OP Accelerator

Function OP-MCAR(A *(CSC)*, B *(CSR)*, C):
  $\mathcal{P} \leftarrow \emptyset$ // $\mathcal{P}$ stores partial sums
  while *matrix* A *has unprocessed nonzeros* **do**
    $S_A \leftarrow \emptyset$
    while $|S_A| < X$ *and matrix* A *has unprocessed nonzeros* **do**
      read an element (i, r, val_A) from A, and mark val_A with "STREAM_A" // STA
      $S_A \leftarrow S_A \cup \{(i, r, val_A)\}$ // i and r are the row and column indices
    MILES_wait_load()
    foreach $r \in \{r \mid (i, r, val_A) \in S_A\}$ **do**
      $cursor[r] \leftarrow B.rowptr[r]$;
      $end[r] \leftarrow B.rowptr[r+1]$ // STR
    while *there exist unfinished* B *rows* **do**
      foreach $r_B \in \{r \mid (i, r, val_A) \in S_A\}$ **do**
        if $cursor[r_B] \geq end[r_B]$ **then continue**
        $bp \leftarrow cursor[r_B]$;
        $cursor[r_B] \leftarrow cursor[r_B] + 1$
        $j \leftarrow B.colidx[bp]$
        foreach $(i, r, val_A) \in S_A$ **do**
          $val_B \leftarrow$ **MS**($\&B.values[bp]$, "STREAM_B")
          $\mathcal{P} \leftarrow \mathcal{P} \cup \{(i, j, val_A \times val_B)\}$
      MILES_wait_load()
  merge $\mathcal{P}$ to obtain output matrix C, and mark writeback requests with "STREAM_C" // WB
  MILES_wait_store()
  MILES_ci_finish()

matrix C in a compressed way with store instructions marked with `STREAM_C`.

VI. Cycle-Level Simulation Framework

MILES adopts CISim [20] as its CPU model. CISim is a trace-driven, cycle-level CPU simulator that runs LLVM IR. The CPU sends MIs to a coprocessor for execution, and the coprocessor uses MCPM to model MAs. MCPM is implemented via a memory request generator (MRG) that reads address streams and replays memory requests.

A. Coprocessor Microarchitecture

The coprocessor models a microarchitecture similar to Gemmini [21], as shown in Fig. 6. Instructions from the CPU are first buffered in a command queue (CmdQ) and then enter the reservation station. The reservation station contains execute, load, and store queues for the corresponding instruction types. The ROB detects data hazards by checking whether SPM address ranges of instructions overlap. Once an instruction is ready, it enters an execute, a load, or a store controller. These controllers are allowed to execute out of order, but instructions within each controller are processed in order. Load and store instructions are served through a DMA engine, which moves data between SPM and L2 Cache.

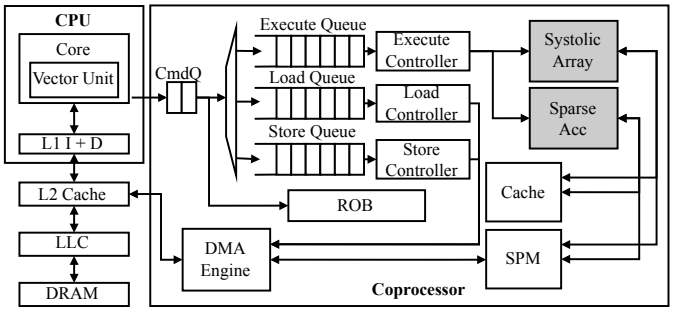


Fig. 6: Coprocessor microarchitecture model

B. Configuration File for MCRA

MILES requires a configuration file for MCAR, which is shown in Fig. 7. In this file, `MILES_functions` specifies functions used to represent custom MIs, `MILES_traces` provides files for address streams in the form of key-value pairs, `order_file` specifies the name of the address stream order file, and `mem_type` is used to specify memory types used by address streams.

```
miles_functions: ["GUSTAVSON_MCAR"]
stream_traces:
    STREAM_A: stream_STREAM_A.txt
    STREAM_B: stream_STREAM_B.txt
    STREAM_C: stream_STREAM_C.txt
order_file: stream_order.txt
mem_type:
    STREAM_A: cache # or spm
    STREAM_B: cache
    STREAM_C: cache
stream_dep_args:
    GUSTAVSON_MCAR: ["spmsrc", "spmdst", "size", "size", "no", ...]
```

Fig. 7: Configuration file format for MCAR

When MIs use SPM, data must be explicitly transferred between DRAM and SPM via DMA instructions. Therefore, MILES needs to construct data dependencies between MI and DMA instructions. MILES currently supports two types of DMA instructions. The `mvin` instruction moves data from DRAM to the SPM, while the `mvout` instruction does the reverse. The configuration includes a field `stream_dep_args` to specify the argument types for MCAR kernels. Specifically, `spmsrc` and `spmdst` indicate arguments that correspond to a data address read (written) from (to) SPM, respectively. `size` is the data size. To build dependency, MILES assigns data sizes to address types in the order they appear. Irrelevant input arguments are labeled as `no`.

C. Matrix Accelerator Simulation Based on MCPM

MILES implements MCPM through a Memory Request Generator (MRG) to model both SAs and SpMSpM accelerators, which differ in how memory requests are generated. MRG directly generates vector memory requests based on the DRAM address, SPM address, and data sizes encoded in MIs for SAs, but MRG generates memory requests based on MCAR kernels.

According to MCPM, the execution time of a data vector consists of a read latency R_i, a compute latency C_i, and a writeback latency W_i. To model these latencies, MRG maintains two timing attributes for each memory request: Earliest Issue Time (EIT) and Delayed Complete Time (DCT). Meanwhile, when a read request's data is fetched, MILES marks it as Data Ready (DR). EIT controls when a memory request can be sent to the memory system, modeling the fact that a writeback request can only be issued after its preceding computation completes. Specifically, if a read request reaches the DR state at time t, MRG generates its corresponding write request at the same time. To model a compute latency of C_i cycles, the EIT of this write request is set to $t + C_i$. DCT is used to model compute latency of data in the STA phase that causes no write requests. For example, if a data request reaches DR at time t, its DCT is set to $t + C_i$. In simulation, MRG maintains two queues: pending and issued memory requests. In each cycle, MRG first issues pending requests that have reached EIT and hardware resources are available to the memory system. MRG then checks the queue of issued requests and marks requests that have reached DCT as completed. DCT can only be determined after a request reaches the DR state, because the simulator cannot predict exactly when the request data will be fetched.

MILES implements GEMM instructions similar to Gemmini [21], including: (i) `preload` that preloads data from SPM to PEs in WS dataflow; (ii) `matmul` and `matmul_out` perform GEMM. The latter is used to write partial sums stored in the array to an accumulator after GEMM in OS dataflow. In MILES, input matrices can only be read from SPM, and partial sums can only be written to the accumulator using `mvin` and `mvout`, respectively. MILES also provides instructions for synchronizing the CPU and coprocessor. MILES implements event-driven simulation for both WS and OS SAs. Assume an SA size is $d \times d$. In WS dataflow, when an input vector reaches the DR state at time t, MILES generates a write request for the corresponding output vector and sets its EIT to $t + 2d - 1$. In OS dataflow, all write requests are inserted simultaneously into the pending queue when the last input vector reaches the DR state. EITs for these write requests are assigned starting at $t + 2d - 1$ and are incremented sequentially.

For MCAR kernels, MRG replays memory requests based on address streams and suspends, synchronizes, and finishes request generation using numerical flags of synchronization primitives. Note that MRG implements synchronization primitives based on DR rather than DCT, i.e., load synchronization (`MILES_wait_load`) completes when all read requests reach DR. In MCAR kernels, only when `delay` is introduced in `MARK_STREAM` does MRG add a delay for EIT.

VII. Experiment

A. Validation Against BOOM+Gemmini

1) Methodology: We validate the accuracy of MILES against RTL simulation of BOOM [22]+Gemmini [21]. The CPU is the large BOOM, while Gemmini uses a 16×16 SA, a 256KB SPM, and a 64KB accumulator. The memory system includes two levels of cache and 2GB of DDR3 memory, as shown in Table IV.

TABLE IV: Memory system configuration

Memory	Configuration
L1 ICache	32KB / 8-way / 2-cycle hit latency
L1 DCache	32KB / 8-way / 4-cycle hit latency
L2 Cache	512KB / 8-way / 10-cycle hit latency
DRAM	2GB DDR3, 500MHz, 160ns latency

Benchmarks contain multiple DNN models, GEMM kernels, and Conv2D kernels. DNNs include ResNet50, MobileNet, and Small/Base/Large BERT. Among them, G1-G4 correspond to four GEMMs with input sizes of 256, 512, 1024, and 2048, respectively. C1 to C4 correspond to four Conv2Ds, whose input feature map sizes are 7×7, 14×14, 28×28, and 56×56, respectively. The number of input and output channels is the same across all configurations (512, 256, 128, and 64), and the kernel size is fixed at 3×3. The experiments are based on the DNN implementations in the Gemmini repository. We replace the Gemmini instructions with MILES SA instructions for simulation.

2) Results: Fig. 8 shows the error rates between MILES and RTL simulation. Since Gemmini does not provide an OS SA implementation for Conv2Ds, their results are not reported. Results show that the average errors of MILES for OS and WS SAs are 9% and 10.5%, respectively. Since we measure execution cycles end-to-end, some errors are attributable to the CPU model. MILES currently lacks modeling of address translation, which can significantly affect accelerator performance [33]. In addition, as shown in Fig. 9a, MILES achieves an average simulation speed of 432× over the 8-thread RTL simulation. For example, RTL simulation of G4 takes 16 hours, whereas MILES only takes 440 seconds. These results demonstrate that MILES can significantly reduce simulation time while maintaining high accuracy.

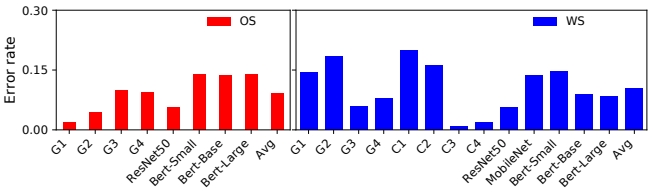


Fig. 8: Accuracy validation of MILES

We further compare the accuracy of MILES with ONNXim [15], mNPUSim [14], and PytorchSim [11]. All simulators use the WS SA, and workloads are G1 to G4. Results are shown in Fig. 9b. The errors of these simulators are 55.1%, 43.8%, and 42%, respectively, whereas MILES's error is only 11.7%. Both the lack of instruction-level OOO CPU modeling and the significant difference in the number of simulated instructions cause large errors for them.

Fig. 9c shows the number of simulated instructions of all simulators. Taking the G4 (2048×2048) as an example, ONNXim and PytorchSim similarly model 2.2 million instructions. mNPUSim models about 14 million events to drive the simulation, whereas the RTL simulation executes about 89 million RISC-V instructions. MIs account for most of the execution time, but other scalar instructions still affect overall

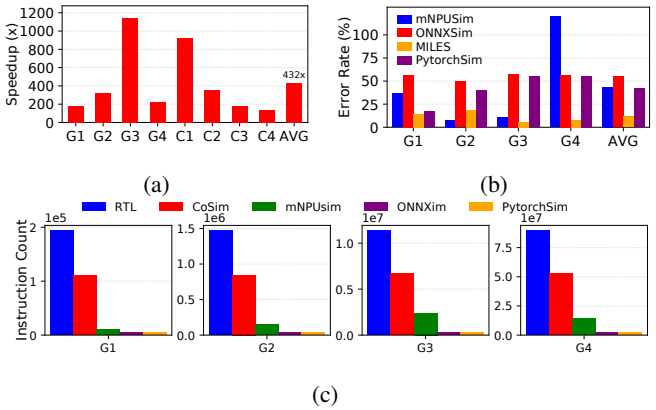


Fig. 9: (a) Simulation speed comparison between MILES and the 8-thread RTL simulator; (b) accuracy comparison among MILES, ONNXim, mNPUSim, and PytorchSim; (c) Comparison of the number of simulated instructions

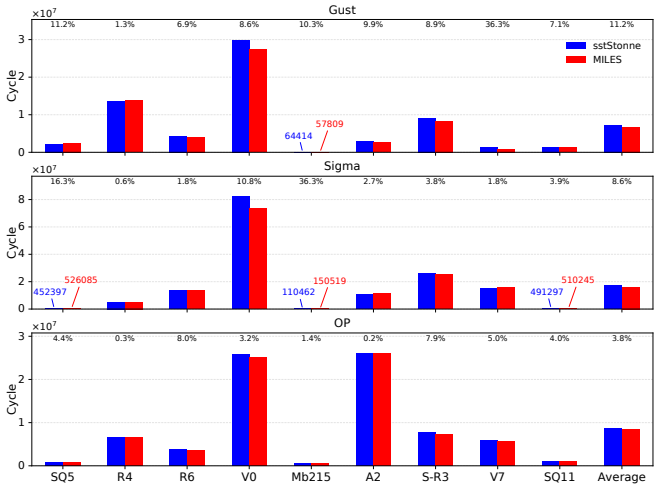


Fig. 10: Validating MILES performance against sstStonne

performance. In contrast, MILES executes about 53 million LLVM IR instructions, which is much closer to the actual count. The main reason for this gap is due to ISA differences between LLVM IR and RISC-V.

B. Validation Against sstStonne

1) Methodology: We validate the accuracy of MILES against sstStonne [23] for SpMSpM accelerators, including Sigma, OP, and Gust. All accelerators are configured with 128 multipliers, and both distribution and reduction bandwidth are 128. The memory system uses a two-level cache hierarchy, as shown in Table IV. All experiments use 32-bit integer precision. Sparse matrices used in experiments are shown in Table V. These matrices are selected from representative layers of various DNN models [10], with matrix A serving as the stationary matrix and matrix B as the streaming matrix.

TABLE V: Sparse matrix dimensions and sparsity

Name	M, N, K	Sparsity of Matrix A (%)	Sparsity of Matrix B (%)
$SQ5$	64, 2916, 16	68	11
$SQ11$	128, 729, 32	70	10
$R4$	256, 3136, 64	88	9
$R6$	64, 2916, 576	89	53
$S\text{-}R3$	64, 5329, 576	89	46
$V0$	128, 12100, 576	90	61
$MB215$	128, 8, 512	50	10
$V7$	512, 144, 4608	90	94
$A2$	384, 121, 1728	70	54

2) Results: Fig. 10 shows validation results. MILES achieves average errors of 11.2%, 8.6%, and 3.8% for Gust, Sigma, and OP accelerators, respectively, indicating that modeling accelerators with MCPM and MCAR is reliable. We also compare the simulation speed. MILES's average simulation speed is 20× faster than sstStonne when simulating Gust. sstStonne models NoCs and PEs in detail, while MCPM simplifies their modeling, thereby reducing simulation time.

MILES is built on a memory-centric performance model, so the accuracy of its memory-system modeling is critical. Fig. 11 compares hit rates of L1 and L2 caches of MILES and sstStonne on different accelerators. On Gust, Sigma, and OP accelerators, L1 Cache hit rate errors of MILES are 0.08%, 0.04%, and 0.017%, respectively, while the L2 Cache hit rate errors are 8.4%, 0.45%, and 8.0%, respectively. Matrices used in experiments are relatively small, so most data can be served by L1 Cache. However, data reuse in L2 Cache is very low, resulting in extremely low L2 Cache hit rates in some cases.

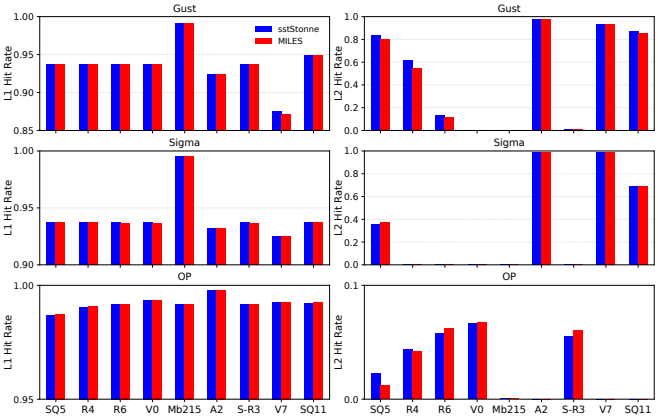


Fig. 11: Validating L1 and L2 cache hit rates of MLIES against sstStonne

VIII. Case Study: Memory Access Optimization for SpMSpM Accelerators

Heterogeneous on-chip memories have been widely adopted in accelerators to handle diverse memory access patterns. For example, Flexagon [10] uses a FIFO for the read-only matrix and a Cache for irregular data, and SPADE [34] introduces a bypassing technique to reduce cache pollution. More general platforms, such as the Sunway many-core processor [35] and GPUs [36], can also configure their on-chip memory as SPM and Cache. In this case study, we use MILES to study how heterogeneous on-chip memories can improve the performance

of SpMSpM accelerators. We select the Gust accelerator and use both SPM and L1 Cache as on-chip memories, where DMA transfers data between the SPM and L2 Cache.

A. Hybrid Buffering Scheme and Instruction Support

To fully utilize heterogeneous memories, we implement a Hybrid Buffering Scheme (HBS), which (1) places matrices accessed regularly and irregularly in SPM and Cache, respectively; (2) prioritizes read-only and write-only data for use in SPM to reduce data transfers when data is too large to be placed in SPM. Placing regular data in SPM not only avoids complex address computations, thereby alleviating the configuration wall, but also avoids interference between irregular and regular data in L1 Cache, which is fully used by the former. To use HBS in the Gust dataflow, matrices A and C are accessed via SPM, and matrix B is accessed via Cache. For simplicity, metadata is ignored. In this case, new MIs should be introduced because both DRAM and SPM addresses need to be encoded explicitly.

In MILES, new instructions can be easily simulated using MCAR. Listing 1 shows four new MIs for Gustavson SpM-SpM of size $M \times N \times K$ and 32-bit integer precision. The algorithm first computes the number of rows of matrix A and C that can be placed in SPM, denoted as $n\_row\_per\_tile$. Since the Gust accelerator compresses output matrix C, it is difficult to determine the number of non-zero elements of C rows prior to computation. To simplify data tiling, we tile the matrices A and C at full-row granularity and reserve SPM space for K and N elements per row, respectively. But nonzero elements of matrix A are moved into SPM according to their actual sizes using `mvin`. After computation, the algorithm uses `mvout` to move compressed C rows stored in SPM back to DRAM.

```
// A: MxK (int32), B: KxN (int32), C: MxN (int32)
size_t dsize = sizeof(int32_t);
int n_row_per_tile = Cap / ((K + N) * dsize);
int C_base = K * n_row_per_tile * dsize;

for (int i = 0; i < M; i += n_row_per_tile) {
   int n_tile = min(n_row_per_tile, M - i);
   int row_start = A->rowptr[i];
   int row_end = A->rowptr[i + n_tile];
   int A_size = (row_end - row_start) * dsize;
   int C_size = N * n_tile * dsize;

   mvin(&A->values[row_start],&SPM[0],A_size,1);
   for (int j = i; j < i + n_tile; j++) {
      GustAState st1 = gust_1(
         &SPM[A->rowptr[j] - row_start],
         (uintptr_t)A->rowptr,
         (uintptr_t)A->colidx,
         (A->rowptr[j+1] - A->rowptr[j]) * dsize);
      GustBState st2 = gust_2(
         (uintptr_t)B->rowptr,
         (uintptr_t)B->colidx,
         (uintptr_t)B->values);
      GustCState st3 = gust_3(
         &SPM[C_base + (j - i) * N * dsize],
         i, j, N,
         N * dsize);
      gust_4(st1, st2, st3);}
   mvout(&C[i * N], &SPM[C_base], C_size, 1);}
```

Listing 1: HBS for Gust Accelerator with new instructions

Due to the limited instruction bit width, a single instruction cannot encode all addresses and parameters of matrices A, B, and C. Therefore, we decompose a SpMSpM into four instructions. For each row in a tile, the algorithm sequentially invokes `gust_1`, `gust_2`, and `gust_3` to configure parameters of matrices A, B, and C, respectively. Here, $GustAState$, $GustBState$, and $GustCState$ are structures used to store operands configured by `gust_1`, `gust_2`, and `gust_3`, respectively. These functions only pack operands into structures and are treated as configuring instructions in simulation. It then invokes `gust_4` to start the accelerator, which is an MCAR kernel. It extracts required variables from these structures and defines accelerator behaviors. `gust_3` also takes i and j as inputs, which indicate the starting row of a tile and the current row to process, respectively. `gust_4` is similar to the Gust dataflow in Algorithm 1, but it only computes a single stationary row. The baseline is similar to Listing 1 but does not require data-movement instructions. During simulation, bodies of all instructions represented as functions are not executed.

B. Evaluation Methodology and Results

1) Methodology: The experiments are conducted using MILES with the same experimental configuration as in Section VII-B1, except changing L1 Cache size to 128KB and adding a 128KB SPM. To compare fairly and avoid performance improvement by enlarging on-chip memories, L1 cache size of the baseline is set to 256KB. The SPM is configured with 128 banks, where consecutive 32-bit addresses are mapped to consecutive banks, supporting up to 128 parallel data accesses in a single cycle. Six SpMSpMs ($N \times N \times N$) are selected, where N is 128, 256, 384, 512, 1024, and 2048. For each size, five sparsity rates are used: 90%, 95%, 99%, 99.5%, and 99.9%, covering typical SpMSpM scenarios from moderately sparse to extremely sparse. Matrices A and B use the same sparsity rate. During random generation, each row is guaranteed to contain at least one nonzero element to avoid all-zero rows when the sparsity is high.

2) Results: Experimental results are shown in Fig. 12. HBS improves performance on most tests, with an overall average speedup of 1.22×. Across different sparsities, HBS performance first increases and then decreases. When matrix sparsity is relatively low, the number of nonzero elements is large, and memory access remains continuous and local. In this case, irregular memory access is not yet critical, so HBS benefits only limitedly. As sparsity increases, memory accesses become more irregular, and invalid data movement between Cache hierarchies and high-latency DRAM accesses due to data misses becomes more significant, so HBS benefits more. However, when matrices are extremely sparse, the amount of data in computation is significantly smaller, and L1 Cache can already hold most of it. In this case, additional DMA overhead introduced by HBS can no longer be hidden and results in a performance degradation.

After applying HBS, L1 Cache hit rate reaches 97.8%, but the improvement over the baseline is only 2%. Therefore, to more accurately analyze the impact of HBS on memory access, Fig. 13 shows memory traffic before and after applying HBS.

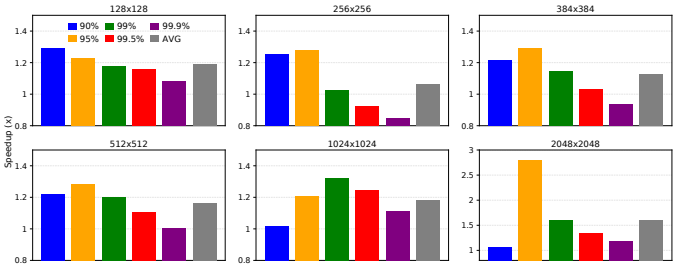


Fig. 12: Speedup achieved by HBS

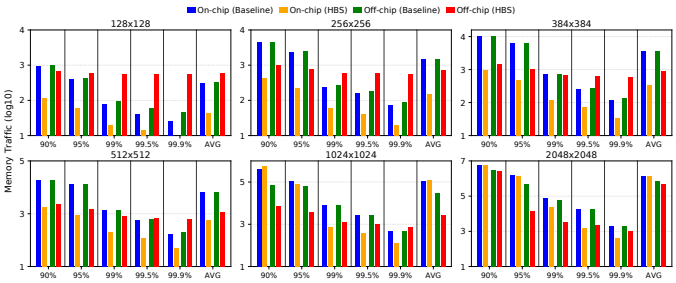


Fig. 13: L1 Cache hit-rate improvement achieved by HBS

Here, on-chip traffic refers to the number of requests sent to L2 Cache due to L1 Cache misses, while off-chip traffic refers to the number of requests that further go to DRAM due to L2 Cache misses. Results show that HBS significantly reduces on-chip traffic in most matrices. HBS maps matrices A and C to SPM, and L1 Cache is dedicated primarily to matrix B, reducing cache contention and decreasing data traffic between L1 and L2 Caches.

Furthermore, although the baseline generates more L2 requests, its L2 Cache hit rate is only 8%. This indicates that L1 misses can hardly achieve effective reuse in L2 Cache, and many requests still need to access DRAM. In contrast, with HBS, L2 Cache hit rate increases to 71.4%. Matrices A and C have relatively regular memory access patterns, and DMA requests are sent directly to L2 Cache, achieving good locality. Compared with the baseline, HBS reduces off-chip traffic for most matrices, thereby mitigating high-latency DRAM accesses of the Gust accelerator. For matrix sizes of 256×256 and 384×384, performance slowdowns caused by HBS occur at extremely high sparsities, such as 99.5% or 99.9%, because of high relative DMA overhead. However, the 128×128 case has similar off-chip traffic to them but still achieves performance improvement. Using SPM reduces available L1 Cache capacity to half of that in the baseline. As the size of matrix B increases, irregular accesses are more likely to exceed L1 Cache capacity, resulting in more L1 Cache misses.

IX. Conclusion

To address the shortcomings of existing NPU simulators, this paper proposes MILES, a simulation framework designed for CPU matrix extensions. MILES introduces a memory-centric matrix accelerator performance model (MCPM), which improves simulation speed while ensuring simulation accuracy, and enables unified modeling of various GEMM and SpMSpM accelerators. MILES also proposes a memory-centric accelerator representation (MCAR), which enables flexible modeling of accelerators via external C-language descriptions. MILES is built on CISim, enabling instruction-level and system-level co-simulation of the CPU, accelerators, and memory system. Experimental results show that MILES achieves simulation errors of only 10.5% and 9% on WS and OS SAs, respectively, with a simulation speed 432 times faster than 8-thread RTL simulation. For three representative SpMSpM accelerators, MILES achieves a maximum error of only 11.2%, achieving a 20-fold speedup compared to sstStonne. Moreover, MILES shows its flexibility and strength for exploring matrix extension designs through a case study.

References

[1] Intel, "Intel 64 and ia-32 architectures software developer's manual," https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[2] J. P. de Carvalho, J. E. Moreira, and J. N. Amaral, "Compiling for the ibm matrix engine for enterprise workloads," *IEEE Micro*, vol. 42, no. 5, pp. 34–40, 2022.

[3] ARM, "Arm sme," https://developer.arm.com/documentation/109246/0101/SME-Overview/SME-and-SME2.

[4] RISC-V International, "Risc-v integrated matrix extension," https://github.com/riscv-admin/integrated-matrix-extension.

[5] ——, "Risc-v vector-matrix extension," https://riscv.atlassian.net/wiki/spaces/VMEX/pages/554991628/Vector-Matrix+Extension+VME+-+PoW.

[6] ——, "Risc-v attached matrix extension," https://github.com/riscv-admin/attached-matrix-extension.

[7] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "Extensor: An accelerator for sparse tensor algebra," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 319–333.

[8] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 58–70.

[9] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging gustavson's algorithm to accelerate sparse matrix multiplication," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 687–701.

[10] F. Muñoz-Martínez, R. Garg, M. Pellauer, J. L. Abellán, M. E. Acacio, and T. Krishna, "Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 252–265.

[11] W. Yang, Y. Shin, O. Woo, G. Park, H. Ham, J. Kang, J. Park, and G. Kim, "Pytorchsim: A comprehensive, fast, and accurate npu simulation framework," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 1363–1380.

[12] S. Xi, Y. Yao, K. Bhardwaj, P. Whatmough, G.-Y. Wei, and D. Brooks, "Smaug: End-to-end full-stack simulation infrastructure for deep learning workloads," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 17, no. 4, pp. 1–26, 2020.

[13] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdhar, and T. Krishna, "Scale-sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2025, pp. 186–200.

[14] S. Hwang, S. Lee, J. Kim, H. Kim, and J. Huh, "mnpusim: Evaluating the effect of sharing resources in multi-core npus," in *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2023, pp. 167–179.

[15] H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, and G. Kim, "Onnxim: A fast, cycle-level multi-core npu simulator," *IEEE Computer Architecture Letters*, 2024.

[16] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj *et al.*, "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.

[17] R. Organization, "Spike risc-v isa simulator," https://github.com/riscv-software-src/riscv-isa-sim.

[18] Y. S. Shao, S. L. Xi, V. Srinivasan, G.-Y. Wei, and D. Brooks, "Co-designing accelerators and soc interfaces using gem5-aladdin," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[19] S. Rogers, J. Slycord, M. Baharani, and H. Tabkhi, "gem5-salam: A system architecture for llvm-based accelerator modeling," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 471–482.

[20] X. Hao, S. Zhang, L. Qiao, J. Shi, J. Chen, and H. An, "Cisim: Isa-agnostic custom instruction simulation for general-purpose processor," in *2026 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2026, pp. 1–7.

[21] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.

[22] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "Sonicboom: The 3rd generation berkeley out-of-order machine," in *Fourth Workshop on Computer Architecture Research with Risc-v*, vol. 5. International Symposium on Computer Architecture Valencia, 2020, pp. 1–7.

[23] F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, and T. Krishna, "Stonne: Enabling cycle-level microarchitectural simulation for dnn inference accelerators," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 201–213.

[24] J. Van Delm, A. Lydike, J. Dumoulin, J. Crols, X. Yi, R. Antonio, J. Woodruff, T. Grosser, and M. Verhelst, "The configuration wall: Characterization and elimination of accelerator configuration overhead," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2026, pp. 265–280.

[25] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz *et al.*, "The rocket chip generator," *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, vol. 4, pp. 6–2, 2016.

[26] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEEInternational Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2019, pp. 304–315.

[27] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 754–768.

[28] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, "Sparseloop: An analytical approach to sparse tensor accelerator modeling," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1377–1395.

[29] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "Scale-sim: Systolic cnn accelerator simulator," *arXiv preprint arXiv:1811.02883*, 2018.

[30] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 2–14.

[31] N. Nayak, T. O. Odemuyiwa, S. Ugare, C. Fletcher, M. Pellauer, and J. Emer, "Teaal: A declarative framework for modeling sparse tensor accelerators," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1255–1270.

[32] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 1–12.

[33] B. Hyun, Y. Kwon, Y. Choi, J. Kim, and M. Rhu, "Neummu: Architectural support for efficient address translations in neural processing units," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 1109–1124.

[34] G. Gerogiannis, S. Yesil, D. Lenadora, D. Cao, C. Mendis, and J. Torrellas, "Spade: A flexible and scalable accelerator for spmm and sddmm," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.

[35] X. Hao, T. Fang, J. Chen, J. Gu, J. Feng, H. An, and C. Zhao, "swMPAS-A: Scaling mpas-a to 39 million heterogeneous cores on the new generation sunway supercomputer," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 141–153, 2022.

[36] NVIDIA, "NVIDIA tesla v100 GPU architecture," https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf, 2017.